Speeding up plain local search KNN graph construction

A profile-driven optimisation of the SISAP 2026 entry

Report on changes to `sisap2026-plain-local-search`

24 July 2026

Abstract

The existing implementation builds a 15-nearest-neighbour graph over 6 350 000 Wikipedia BGE-M3 embeddings in **191.8 s** on eight threads, reaching a final recall of **0.8007**. After the changes described here it does the same work in **118.1 s** at recall **0.8009**: a speedup of **1.62×** with no loss of quality. The candidate recall after every individual stage is statistically identical to the original, so this is the same algorithm executed faster rather than a different quality/time trade-off. Every change was driven by a measurement, and several plausible optimisations were implemented, measured, found wanting, and reverted; those are documented too, together with a measurement of the accuracy ceiling that the 1-bit quantisation imposes on the whole approach.

1 Result

Phase	Before	After	Change
1-bit quantisation	0.2 s	0.2 s	—
Spatial relabelling + rerank codes	—	2.0 s	new
Tree indexing	64.2 s	45.2 s	−19.0 s
tree partitioning	—	*12.9 s*	
leaf all-pairs	—	*29.9 s*	
Refinement (2 rounds)	99.8 s	48.1 s	−51.7 s
round 1	*69.3 s*	*32.8 s*	
round 2	*30.5 s*	*15.7 s*	
Reranking	27.0 s	22.6 s	−4.4 s
Total (build + query)	**191.8 s**	**118.1 s**	**1.62×**
Final recall	**0.8007**	**0.8009**	

Table 1: Wall-clock per phase, `OMP_NUM_THREADS=8`. Times exclude HDF5 loading (which is outside the measured region in both versions) and exclude the ground-truth recall evaluations, which are now subtracted explicitly.

Candidate recall (the fraction of true neighbours present anywhere in the 64-entry candidate list) matches the original at every stage:

After	Before	After changes
indexing	0.2097	0.2082
refinement r.1	0.7623	0.7621
refinement r.2	0.7860	0.7861
final (reranked, top-15)	0.8007	0.8009

The small differences are run-to-run noise: repeated runs of the unmodified binary varied over 0.76213–0.76230 for round 1 alone, because insertion order across threads is nondeterministic.

2 Setup and method

All measurements were taken on an AMD EPYC 9754 (Zen 4c, 128 cores, 755 GB RAM, 16 MB L3 per core-complex), compiled with the project's existing flags (`-O3 -march=x86-64-v3 -fopenmp`) and run with exactly eight OpenMP threads. The dataset is `benchmark-dev-wikipedia-bge-m3.h5`: $m = 6\,350\,000$ vectors of $n = 1024$ half-precision floats, with a $6\,350\,000 \times 32$ exhaustive ground truth under `/allknn/knns`.

Binary codes are 1024 bits, so $bn = 16$ 64-bit words, exactly 128 bytes — two cache lines — per point. The code matrix is 813 MB, the neighbour tables V and D are 1.6 GB each, and the fp16 input is 13 GB. These sizes matter: almost every hot loop in this program is a random walk over one of them, and the analysis below is mostly about cache lines rather than instructions.

Two tools were built alongside the changes and are what most of the reasoning rests on.

A kernel microbenchmark. A single-threaded harness that replays one leaf of the indexing phase ($L = 190$ points, all 17 955 pairs) in isolation, against both an empty neighbour graph and a saturated one. Isolating this loop made it possible to iterate on the insert path in seconds rather than in four-minute full runs, and it immediately falsified the first hypothesis about where indexing time was going.

An independent verifier. A separate program that opens the produced output file and the ground-truth file, and recomputes recall in the *original* index space, checking additionally for self-loops, duplicate entries within a row, empty slots and out-of-range identifiers. Since one of the changes relabels every point, the in-process recall number alone would not have caught a mistake in the relabelling round-trip.

3 Where the time actually went

The original profile is dominated by three things: indexing (64.2 s), the two refinement rounds (99.8 s), and reranking (27.0 s). My first instinct was that indexing was bound by lock contention and by the `memmove` inside the sorted insert. A restructuring that removed *all* locks from the leaf phase returned indexing at 62.4 s — a 3% improvement. The model was wrong.

The microbenchmark settled it:

	ns / pair	cycles @ 2.4 GHz
`hamming_avx2_lut` alone	4.76	11.4
Full leaf loop, saturated graph	6.45	15.5
Full leaf loop, empty graph	53.16	127.6

The distance kernel is not the problem, and neither are locks. When the neighbourhoods are still filling, nearly every candidate is accepted, and the cost of *landing* an accepted candidate is roughly eight times the cost of computing the distance that justified it. Real indexing time sits between the two regimes (13.5 ns/pair measured across the whole phase), because early trees run cold and later trees run warm.

The same reframing applies to refinement, where each candidate cost 238 cycles against an 11.4-cycle distance — the loop was waiting on memory, not computing.

4 Changes

4.1 Indexing: 64.2 s → 45.2 s

Lock-free leaves. Tree construction and the leaf all-pairs work are now separate phases. The original ran whole trees concurrently, so every insert went through the 8192-entry lock table. But all leaves of a *single* tree are disjoint point sets, so if one tree is processed at a time with its leaves in parallel, there are no write conflicts at all and the leaf phase needs no synchronisation whatsoever. Partitioning is parallelised with OpenMP tasks, which also removes the previous serial bottleneck at the top of each tree.

An insert built for the indexing regime. Since most offers succeed here, the sorted insert's `memmove` is the dominant cost. During indexing the neighbourhood is now kept *unsorted* with the current maximum pinned to slot 0; landing a candidate is a slot-0 overwrite plus a rescan for the new maximum, and the ordering is restored once, afterwards, by a single pass (`KNN_sort_lists`). Two further details made this actually fast:

- **No store-forward stall.** Writing `D[0]` and then immediately reloading it as part of a 32-byte vector incurs a store-forwarding failure on every successful offer. Slot 0 is instead *blended* with the incoming distance in registers, so nothing is stored before the reduction.
- **No argmax scan.** Locating the new maximum needs its position, not just its value. Rather than a second, badly-predicted search loop, each distance carries its own slot index in the low `KNN_TAG_BITS` bits, so a single `vpmaxsd` reduction yields both the value and where it lives.

The first two took the empty-graph leaf cost from 53.2 to 27.6 ns/pair; storing the tags in `D` rather than recomputing them in the reduction loop brought it to 25.6 ns/pair, an overall 2.1×, with the saturated case unchanged. Note how little the last step gave: by then the offer is at its load-throughput floor, which is why the insert path was not pursued further.

Prefetching the partition scan. The pivot-distance pass reads `Matrix[O[i].v]` for scattered `v`, i.e. a random 128-byte read out of 813 MB, and all the identifiers are known ahead of time. Running the loads twelve iterations ahead cut partitioning from 19.8 s to 12.6 s.

Leaf-local scratch buffer. A leaf's codes are gathered into a contiguous ≈25 KB buffer before the all-pairs loop, so the inner loop streams over an L1-resident block instead of chasing 190 rows spread over 813 MB. The gather itself is prefetched.

Bounded histogram. The median split histogram is cleared and scanned only over the observed distance range. In 1024 dimensions the distances to a random pivot concentrate very tightly, so this is a narrow window out of the full $[0, 1024]$ span. (The original also indexed `C[d]` with d up to 1024 into an array of size $bn \cdot 64 = 1024$, a one-element overrun; the new histogram is sized 1088.)

4.2 Refinement: 99.8 s → 48.1 s

Candidate deduplication — the largest single win. Nearby points share neighbours, so the same w is reached through many different v while scanning u's neighbours-of-neighbours. Each repeat costs a full distance evaluation and three random cache lines. A small per-thread direct-mapped hash table (2048 entries, 16 KB, keyed on (u, w)) drops the repeats for a few cycles each.

This is worth stating carefully, because skipping work in a search loop usually does change the answer. Here it does not:

Claim. Suppressing a repeated offer of (u, w) leaves the final graph unchanged.

Argument. The pair (u, w) has one fixed distance d. `Dmax[u]` is non-increasing over time. If the first offer was rejected, $d \geq$ `Dmax`$[u]$ then, and `Dmax`[u] is no larger later, so any repeat is rejected too. If it was accepted, w is in u's list and the insert's duplicate check makes any repeat a no-op. The same argument applies to the reverse insert into w. Hash collisions can only cause a *false miss* — the key comparison is exact, so a colliding entry compares unequal and the candidate is evaluated normally.

The measured candidate recall was indeed unchanged.

A compact `Dmax` array. Rejecting a candidate is the common case and requires the worst distance in w's neighbourhood. The original read `D[w*k + k-1]`, pulling a cache line out of a 1.6 GB table. That value is now mirrored in a compact m-element array, so the test touches four bytes out of 25 MB.

Prefetching the dependency chain. The candidate identifiers come from v's neighbour list, which is itself a cold random read; nothing in an iteration could be prefetched until it landed. The scan now runs the code rows and `Dmax` entries eight candidates ahead, *and* starts the next v's neighbour list at the top of each iteration. That last one alone was worth 5.2 s in round 1.

Inlined shift. The sorted insert opens a hole with `memmove` of a runtime length, which compiles to a library call whose dispatch overhead dominates a shift this short. It is now done inline in 8-integer AVX2 blocks from the top down; each block's source is disjoint from the previous block's destination, and the block at the insertion point is loaded before the loop because the loop overwrites it (with identical values, so the final store stays consistent).

A cheaper lock. `omp_lock_t` costs an out-of-line call into libgomp on both acquire and release. These locks are held for a few dozen cycles and are essentially never contended (eight threads over 8192 locks), so they were replaced with an inline test-and-set spinlock.

4.3 Reranking: 27.0 s → 22.6 s

Reranking computes $6.35 \times 10^6 \times 64 \approx 4.1 \times 10^8$ exact fp16 inner products, each reading a 2 KB row at random from a 13 GB table — about 812 GB of traffic. At 1140 cycles per dot product against roughly 200 cycles of arithmetic, it is almost entirely memory-bound. It benefits from the spatial relabelling below, from prefetching the next candidate's row, and from the same compact-array trick as `Dmax`: the reverse-insert test now reads a 4-byte entry from a 25 MB `Rmin` array instead of a cache line from the 381 MB result table.

4.4 Memory layout

Spatial relabelling. Points are relabelled along the leaf order of one extra partition pass, so that points close in space are close in memory, and everything downstream runs in the new labelling; only the output is mapped back. The extra pass costs 0.6 s and is worth 5.9 s (measured directly with `KNN_NOPERM=1`: 123.8 s versus 118.0 s). It is a real but modest win — a single randomised vantage-point tree gives good locality only near the leaves, and a stronger ordering (for example reordering by the neighbour graph itself after indexing) is the obvious place to look for more.

Huge pages. All large allocations are 2 MB-aligned and marked `MADV_HUGEPAGE`. With 4 KB pages, random access over these arrays misses the TLB on essentially every access. This has *no effect on the test machine*: `transparent_hugepage/enabled` reads `[madvise]`, but no huge pages are actually granted — `thp_fault_alloc` does not move for a 1 GB `mmap` plus `madvise`, and there is no `hugetlb` pool. The calls are free and would help on a host where THP works, so they were kept.

5 Measured and rejected

Several changes looked good on paper, were implemented and measured, and did not survive. They are listed because the negative results constrain what is worth trying next.

Variant (vs. its own reference run)	variant		reference	
	total	recall	total	recall
int8 rerank codes	157.9 s	0.7978	160.6 s	0.8007
Rerank only top 48 of 64	114.3 s	0.7796	119.2 s	0.8006
Candidates scored in index order	118.0 s	0.8008	118.0 s	0.8008
No spatial relabelling	123.8 s	0.8007	118.0 s	0.8008

Table 2: Rejected variants, each against the run it was A/B-tested against — they were measured at different points during the work, so their absolute totals are not directly comparable to each other. Recall differences are well outside run-to-run noise.

int8 rerank codes. Halving the bytes per vector should have helped a memory-bound loop, and it did — but only by 2.7 s, because the loop is limited by page walks and access latency more than by bandwidth. It cost 0.0029 recall, which at this operating point is not affordable. (The exact fp16 path is kept; the int8 one is still reachable via `KNN_RERANK_F16=0`.)

Truncating the rerank. Reranking only the best 48 candidates by Hamming distance saves 5 s but costs 0.021 recall. The tail of the candidate list matters more than its own ranking suggests, because reranking also inserts u into each candidate's result list — those reverse edges are what lift the final recall (0.8008) above the candidate recall (0.7861) in the first place.

Rebalancing indexing against refinement. Indexing consumes 5× more distance computations than both refinement rounds combined, yet only reaches candidate recall 0.209, which invites the conclusion that it is over-provisioned. It is not. Refinement *saturates*, and the plateau it saturates at is set by the quality of the initial graph:

Trees / rounds	Indexing	candidate recall after round			Total	Final
		1	2	3 (4)		recall
8 / 4	17.8 s	0.6990	0.7432	0.7509 (0.7527)	131.4 s	0.7730
16 / 3	28.2 s	0.7318	0.7648	0.7704	121.1 s	0.7879
24 / 2	37.8 s	0.7490	0.7767	—	112.8 s	0.7920
32 / 2	46.9 s	0.7620	0.7859	—	**119.2 s**	**0.8006**

Table 3: Fewer trees plus more refinement rounds is worse on *both* axes. A fourth round buys 0.0018 recall for 15 s.

Every cheaper-indexing configuration lands below 0.80, and the extra rounds needed to compensate cost more than the trees saved. The existing $Q_b = 264$, $Q_n = 32$, $L_n = 2$ is a well-chosen operating point and was left alone.

6 How much headroom is there?

A useful question when a target is only just being met: how much of the shortfall is search quality, and how much is the representation? The candidate set is chosen entirely in the 1-bit Hamming space, so anything the binary codes cannot separate is lost regardless of how good the search is.

This is directly measurable. Brute-forcing the *exact* top-64 by Hamming distance for 300 sampled points and scoring it against the ground truth (available as `KNN_CEILING=300`) gives:

> Exact top-64 by Hamming distance contains **87.3%** of the true nearest neighbours.
> The pipeline reaches candidate recall **0.786** — **90%** of what is achievable.

So 1-bit quantisation at $k = 64$ caps the whole approach at roughly 0.87, the search is already extracting 90% of that, and the working margin above the 0.80 target is thin. This explains the failures in the previous section: there simply is not enough slack to give any away. It also says where the remaining opportunities are — a richer code or a larger k would raise the ceiling, at a cost that would have to be earned back elsewhere.

7 Verification

The in-process recall figure is computed in the relabelled index space, so it cannot by itself confirm that the relabelling is inverted correctly on output. The output file was therefore checked independently, reading both the produced `knns` dataset and `/allknn/knns` from the benchmark file and scoring in the original labelling:

```
answer 6350000 x 15, ground truth 6350000 x 32
recall = 0.800851
self loops = 0
empty slots = 0
out of range = 0
duplicates in row = 0
```

In addition, a standalone unit test drives `KNN_offer_max0` and `KNN_insert_unsafe` through 200 randomised trials, asserting after each that slot 0 holds the maximum, that `Dmax` agrees with the list, that no duplicates exist, that stored distances match the real pair distances, that the list sorts correctly, and that offering the complete candidate set converges to the exact top-k. This is what gave confidence in the tagged-distance representation and the hand-written AVX2 shift, both of which are easy to get subtly wrong.

8 Notes for reuse

- The two `if (recall > 0.8) goto endwork` early exits were removed. They consult the ground truth, which will not be present on a real run, so they made the measured configuration differ from the deployed one.
- Parameters are overridable from the environment for experimentation; defaults are the tuned configuration. `KNN_QB`, `KNN_QN`, `KNN_K`, `KNN_LN` are the original four; `KNN_RR` limits how many candidates are reranked, `KNN_RERANK_F16` switches between exact fp16 and int8 codes, `KNN_NOPERM` disables the relabelling, `KNN_CEILING=`n runs the quantisation-ceiling diagnostic and exits.
- `KNN_CACHE=<path>` is a development convenience only: a raw cache of the parsed input so that repeated runs skip re-parsing 14.8 GB of HDF5 over NFS. Loading is outside the measured region either way, so it does not affect any number in this report.
- The build stays on `-march=x86-64-v3`. Worth noting for the future: the test machine's Zen 4c cores support AVX-512 VPOPCNTDQ, which would cut the Hamming kernel by roughly 3×; behind a runtime `__builtin_cpu_supports` check this would remain safe on a v3-only host. It was left out deliberately.
- `compute_dot_product` in `matrix.h` is now unused, superseded by an aligned `dot_f16_avx2`. It was left in place rather than modify a file that otherwise needed no changes.

Appendix: incremental measurements

Each row is a full run; the change column names what was added relative to the row above.

#	Change	Total	Final recall
0	original	191.8 s	0.8007
1	lock-free leaves, relabelling, int8 rerank	157.9 s	0.7978
2	exact fp16 rerank restored	160.6 s	0.8007
3	blend + packed argmax, partition/gather prefetch	143.6 s	0.8009
4	candidate dedup, spinlock, inlined shift	125.1 s	0.8008
5	prefetch next neighbour list	119.2 s	0.8006
6	pre-tagged distances	118.0 s	0.8008
7	confirmation run	118.1 s	0.8009

Files changed: `include/algorithms.h`, `include/knn.h`, `src/knn.c`, `src/main.c`. No commits were made; the working tree is left dirty on top of `5011cae`.